

## LESSON GUIDE

# Technology Plus

## Unit 2: Programming

### 4.4.1 – Programming Organization

#### Lesson Overview:

In this lesson, students will practice planning a program by starting with one problem in small groups, then exchanging problems with another group to continue development.

#### Students will:

- Discuss different organizational and logical strategies for program development.
- Decompose a problem and construct multiple models of solutions.
- Collaborate on program development.

**Guiding Question:** How do we plan a programming project, especially when working across different teams?

**Suggested Grade Levels:** 8-12

**Technology Needed:** Yes

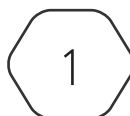
**Approximate Time Required:** 60-75 minutes

#### Standards:

##### CompTIA Tech+ FC0-U71 Objective

- 4.4 – Identify programming organizational techniques and logic concepts.
  - Organizational techniques
    - Pseudo code concepts
    - Object-oriented methods
    - Comments and documentation
    - Flow chart concepts
      - Sequence
  - Logic concepts
    - Branching
    - Looping

*This content is based upon work supported by the US Department of Homeland Security's Cybersecurity & Infrastructure Security Agency under the Cybersecurity Education Training and Assistance Program (CETAP).*



**CYBER.ORG**  
THE ACADEMIC INITIATIVE OF THE CYBER INNOVATION CENTER

Copyright © 2025 Cyber Innovation Center  
All Rights Reserved. Not for Distribution.

# Lesson Background

## Background Information:

This lesson introduces students to foundational organizational strategies in programming. Students will work in groups to begin developing a solution for a problem with a flowchart, then they will exchange flowcharts with another group to develop pseudocode for a new problem. To simulate a work environment, groups will only be given the flowchart and not the original problem of the group they exchange with. This will emphasize for students that working as a computer programmer, or any professional who will be working with code, requires an ability to understand what has already been written before they can write new parts of a codebase, which also requires thoughtful documentation throughout the development process.

## Materials Needed:

|                       |                                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Materials per Class   | <ul style="list-style-type: none"><li>• Lesson Guide</li><li>• Lesson Slides 4.4.1 – Programming Organization</li><li>• Activity Slides 4.4.1 – Planning Programming Solutions</li><li>• Activity Problem Set 4.4.1 – Programming Organization (precut)</li><li>• Specific Teamwork Handout A (precut)</li><li>• Specific Teamwork Handout B (precut)</li></ul> |
| Materials per Group   | <ul style="list-style-type: none"><li>• Activity Handout 4.4.1 – Programming Organization (printed single-sided)</li></ul>                                                                                                                                                                                                                                      |
| Materials per Student | <ul style="list-style-type: none"><li>• Student Notes 4.4.1 – Programming Organization</li><li>• Student Handout 4.4.1 – Programming Organization</li><li>• Assessment 4.4.1 – Programming Organization (optional)</li></ul>                                                                                                                                    |

## Cyber Terms:

| Term       | Definition                                                                                                          |
|------------|---------------------------------------------------------------------------------------------------------------------|
| sequence   | the logical flow of a program; a set of steps to be followed in order                                               |
| flowchart  | a diagram visually representing how a problem will be solved                                                        |
| looping    | a statement that causes one or more other statements to be executed repeatedly                                      |
| branching  | a change in program sequence based on a logical test or condition                                                   |
| pseudocode | a way of writing out code logic in plain, human-readable language before translating it into a programming language |

## 4.4.1 – Programming Organization

**Guiding Question:** How do we plan a programming project, especially when working across different teams?

---

### Lesson Launch (10-15 minutes)

- Tell students that before we start coding on a computer, we need to practice thinking like a computer. We also need to practice communicating with others during the problem-solving process. For this activity, each group will act as a programming team working across the country, just like in a real company.
- Distribute Specific Teamwork Handout A (one image on a 6x10 grid) to half of the class and Specific Teamwork Handout B (one blank 6x10 grid) to the other half. Pair up one Handout A student with one Handout B student so they are sitting back-to-back. The student with Handout A will give instructions to the student with Handout B. The student with Handout B will attempt to draw what student A is describing. Student B may only say, “repeat,” “slow down,” or “wait.” This warmup activity will help students think about how important clear and precise communication is when collaborating.
- Learning programming organization may feel time consuming at first, but it will save us time in the future by helping us clearly define the problem, plan for solutions, and record comments on our thought process all before coding and testing solutions. Programming organization can also help in the debugging process if we have adequately documented our process and used meaningful names for variables, methods, and objects.
- We are going to focus on two types of thinking to build clear, functioning code: organizational techniques and logic concepts

### Organizing Your Coding Plans (20-30 minutes)

- Distribute Student Handout 4.4.1 – Programming Organization and encourage students to complete the handout as you go through today’s lesson.

#### The Planning Process

- *Organizational techniques: help plan, structure, and document what a program is doing.*
  - Problem Identification: Identify the problem you are trying to solve.
  - Visualize the Solution: Draw out how the problem will be solved (flowchart).
  - Turn Visuals into Words: Write out how the flowchart solves the problem (pseudocode).
  - Document your Work: Comment any sections that another user might not understand just from the code/pseudocode.
  - Object-Oriented Programming: Use classes and objects to structure reusable, modular code.
- *Logic concepts: help control how a program behaves – what it does, when it does it, and whether it repeats.*
  - Looping: repeats a block of code until a condition is met
  - Branching: chooses different paths in the code based on conditions

## Identify the Problem

- Step one in the organization plan is to identify the problem. This follows our previous lesson on the basics of computing because we want to:
  - Identify the inputs and how the user enters them.
  - Determine what processes need to happen and in what order.
  - Identify any storage needs like storing variables or accessing existing code.
  - Define what the outputs are and how they will be delivered.
- Read the example we'll be using throughout the main lesson today. Ask students to identify the input(s), process(es), storage, and output(s).
- **Input:** user's age
- **Process:** compare user's age to different movie rating requirements – under 13 is G (General Audiences), under 17 is PG or PG-13 (Parental Guidance Suggested and Parental Guidance Strongly Recommended), and 17+ is R (Restricted)
- **Storage:** store the user's age as a variable; store the Motion Picture Association (MPA) ratings
- **Output:** appropriate MPA rating based on user's age

## Looping

- Discuss when the sample flowchart introduced looping.
- Introduce the two main loop types: for and while. Ask students which one is likely to be used in the Movie Example.

A while loop is more likely because the program will keep asking for the user's age until a valid input is entered.

## Branching

- Discuss when the sample flowchart introduced branching.
- Ask students to write out the decisions that happen in the Movie Example.

Age < 13 (age is less than 13)

13 ≤ Age < 17 (age is greater than or equal to 13 and less than 17)

17 ≤ Age (age is greater than or equal to 17)

## Pseudocode

- Converting flowcharts into human-readable, 'almost code.'
- Uses language that describes the algorithm needed without necessarily using actual commands from any specific programming language.
- Uses keywords and operators.
- Consider having students write their own pseudocode for the Movie Example either from their own flowchart or the sample solution.

## Commenting and Documentation

- In addition to mapping out program development, an important part of the planning process involves documenting the maps in a way that makes them understandable to others.
- Many programming languages use a special symbol for adding comments to code.
- Examples include the hash symbol (#) in Python, Ruby, Shell, and R (a language used for statistics and data analysis); and a double slash (//) in Java, C/C++, JavaScript, and Swift. Several languages use apostrophes (') and asterisks (\*) for longer comment blocks.
- Consider having students add comments to their flowchart or pseudocode for the Movie Example before showing a sample solution of the program with comments.

## Object-Oriented Methods

- Remind students about Object-Oriented Programming (OOP) languages (Lesson 4.1.1) and Objects (Lesson 4.3.4).
- Emphasize the organizational structure of OOP which allows us to abstract everyday concepts into common object classes.
- An easy example to work with prior to the Movie Example is developing a Class called Human . Ask students what properties humans have, even if they have different definitions for different humans (height, age, hair color, etc.). Ask students for examples of human actions (walk, work, play, etc.).
- Guide students through the Movie Example for building an object-oriented model of a movie auditorium.

## Activity: Planning Programming Solutions (20-30 minutes)

- Divide students into groups of 2-3. Distribute one Problem to each group.
- Groups should create a flowchart that maps out a possible solution to their problem, using appropriate symbols.
- Once groups have completed a flowchart, they should exchange flowcharts with another group. Groups should not exchange the original problem description!
- With a new flowchart in hand, groups will write pseudocode for the new problem solely based on the flowchart. They should write the pseudocode only from what information is provided and avoid asking the original group for clarifications.

## Putting It All Together (2-5 minutes)

- Organizing our thoughts before diving into code can help with everything from planning out program requirements, specifying control structures and decision points, and documenting the process for future readers to understand.
- Which organizational structure helps you communicate to future developers about your existing code?

Commenting or documentation.

- What is a benefit of creating a flowchart before writing code?

It helps us visualize a solution before thinking about specific syntax and programming-specific requirements.

- Read the following pseudocode. Explain the program and determine the correct output when num1 = 5 and num2 = 7:

ASK user to input two numbers

if num1 > num2, then DISPLAY num1

else if num2 > num1, then DISPLAY num2

else if num1 = num2, then DISPLAY "EQUAL!"

The user will input two numbers and the program will compare those numbers. If the first number is larger than the second number, the program will display the first number. If the second number is larger, the program will display the second number. If the numbers are equal, the program will display the exclamation, "EQUAL!"

For num1 = 5 and num2 = 7, the program will display 7 because 7 is greater than 5.